UNIVERSITI KUALA LUMPUR

MALAYSIAN INSTITUTE OF INFORMATION TECHNOLOGY

L01-B04

LAB WEEK 4 :

METADATA

BIL	NAME	MATRIX NUMBER
1	MUHAMMAD SYAKIRIN BIN MOHD HANIF	52215125427

INSTRUCTOR: NOR ADANI KAMAL MOHAMAD NASIR

Introduction

In the Vulnerability Analysis, file's extension such as .exe, .jpg, .pdf can't be trusted as proof of it's actual contents. Malicious attacks often disguise payloads by altering file extension or embedded a hidden data within the file.

The objective of this lab's exercise is to perform a systematic investigation of a provided .zip archive which containing various suspicious files. For this lab, we can use Linux and specialized command tools, so this report will document the method to extract the hidden metadata, the verification of true file and disclose the embedded payloads within the files.

Methodology & Tools

In order to analyses the provided files and bypass the file extensions, we need to use a suitable diagnostic and extraction tools given by this task.

1. *File command*

The **file** command is a standard Unix program which often used to determine file type. Instead of relying on the file extension, it reads the file's header to identify it's original format, immediately disclose the disguised or misnamed files.

2. *Hexeditor & Strings*

A hexeditor enable user to view the raw hexadecimal data and ASCII text of file. The **strings** command automates the extraction of human-readable text from original binary files. These tools are utilized to validate the file signature such as JPEG or JFIF and captured readable flags from the raw data.

3. *Exiftool*

It is a versatile command in Linux which used to read, write and manipulate the image, audio or video data. In this lab, it is utilized to inspect the internal tags of image files such as EXIF data to disclose the hidden text or any flags in the files.

4. *Binwalk*

One of the powerful tool which created to searching a given binary image for embedded files and executable codes. It is primarily to identify and collect data that has been hidden in the other file format, serving a useful too; to detect file inception cases.

Execution of The Task

Target 1 : Ocean.jpg

image.zip > **image**

Name	Date Modified	File Size
dog.jpg	2025-10-28	86.3 KB
.DS_Store	2026-03-31	6.00 KB
solitaire.exe	2025-10-28	116 KB
rubiks.jpg	2025-10-28	143 KB
computer.jpg	2025-10-28	575 KB
ocean.jpg	2025-10-28	41.9 KB

Figure 1 image.zip files

- Objective : To inspect the image for hidden text
- Tool Used : exiftool
- Command : exiftool ocean.jpg

≥ POC :

```
ExifTool Version Number         : 13.25
File Name                       : ocean.jpg
Directory                       : .
File Size                       : 43 kB
File Modification Date/Time     : 2025:10:27 12:41:41-04:00
File Access Date/Time           : 2026:03:30 01:46:31-04:00
File Inode Change Date/Time     : 2026:04:12 02:32:45-04:00
File Permissions                : -rw-r--r--
File Type                       : JPEG
File Type Extension             : jpg
MIME Type                       : image/jpeg
JFIF Version                    : 1.01
Resolution Unit                 : inches
X Resolution                    : 72
Y Resolution                    : 72
Profile CMM Type                : Little CMS
Profile Version                 : 2.1.0
Profile Class                   : Display Device Profile
Color Space Data                : RGB
Profile Connection Space        : XYZ
Profile Date Time               : 2012:01:25 03:41:57
Profile File Signature          : acsp
Primary Platform                : Apple Computer Inc.
CMM Flags                       : Not Embedded, Independent
Device Manufacturer             :
Device Model                    :
Device Attributes               : Reflective, Glossy, Positive, Color
Rendering Intent                : Perceptual
Connection Space Illuminant     : 0.9642 1 0.82491
Profile Creator                 : Little CMS
Profile ID                      : 0
Profile Description             : c2
Profile Copyright               : IX
Media White Point               : 0.9642 1 0.82491
Media Black Point               : 0.01205 0.0125 0.01031
Red Matrix Column               : 0.43607 0.22249 0.01392
Green Matrix Column             : 0.38515 0.71687 0.09708
Blue Matrix Column              : 0.14307 0.06061 0.7141
Red Tone Reproduction Curve     : (Binary data 64 bytes, use -b option to extract)
Green Tone Reproduction Curve   : (Binary data 64 bytes, use -b option to extract)
Blue Tone Reproduction Curve    : (Binary data 64 bytes, use -b option to extract)
Comment                         : THIS IS THE HIDDEN FLAG
Image Width                     : 640
Image Height                    : 425
Encoding Process                : Progressive DCT, Huffman coding
Bits Per Sample                 : 8
Color Components                : 3
Y Cb Cr Sub Sampling            : YCbCr4:2:0 (2 2)
Image Size                      : 640x425
Megapixels                      : 0.272
```

Figure 2 Exiftool Result

≥ Analysis :

By utilizing Exiftool to perform a deep scan of the image's metadata, we have successfully bypassed the visual layer of the file in order to examine the EXIF data which stands for Exchangeable Image File Format. In this instance, a manual review of the Exiftool ouput has successfully revealed a hidden flag which embedded within the comment field. Attackers or creators often use this to hide the secret flags or unauthorized text without altering the visual appearance of the images.

Target 2 : Computer.jpg

image.zip > **image**

Name	Date Modified	File Size
dog.jpg	2025-10-28	86.3 KB
.DS_Store	2026-03-31	6.00 KB
solitaire.exe	2025-10-28	116 KB
rubiks.jpg	2025-10-28	143 KB
computer.jpg	2025-10-28	575 KB
ocean.jpg	2025-10-28	41.9 KB

Figure 3 image.zip files

- Objective : To verify the structural integrity and real identity of the file
- Tool Used : hexeditor & strings
- Command : **hexeditor computer.jpg** and **strings computer.jpg**

≥ POC :

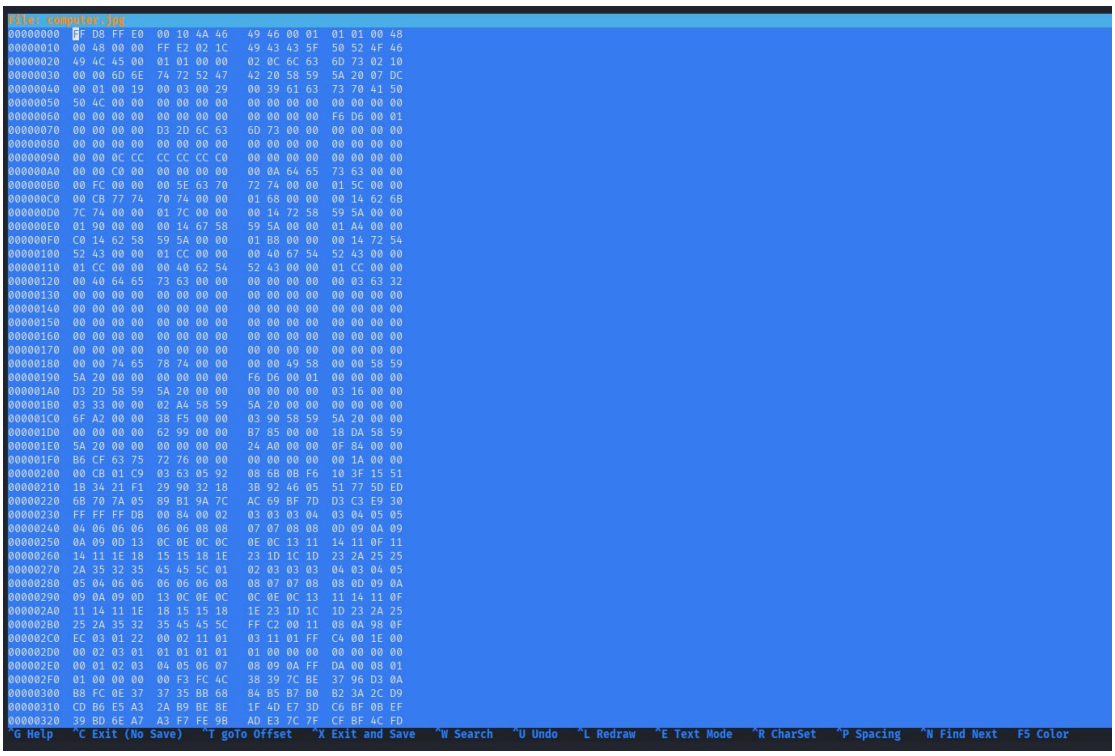

Figure 4 Hexeditor Analysis Result

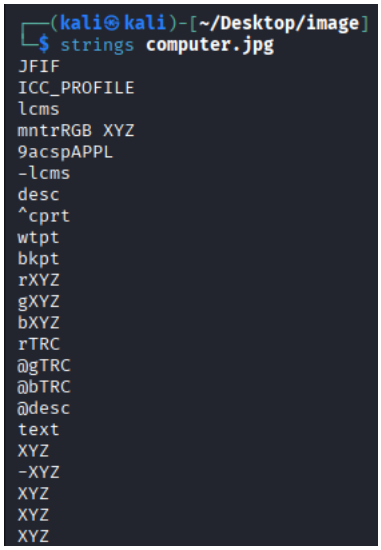

Figure 5 Strings Extraction Result

≥ Analysis :

To check a file's extension is never promising in digital investigation. Therefore, proving the integrity and true identity is a must, so we use hexeditor to dive straight into the raw binary code. By examining the file header, the first line of bytes of data show the magic bytes of **FF D8 FF E0**. When translated to ASCII, this corresponds to the JFIF which stands for (Jpeg File Interchange Format), JFIF. This prove the doubt that **computer.jpg** is a legitimate, harmless JPEG files and not a disguise payload.

Target 3 : Solitaire.exe

image.zip > **image**

Name	Date Modified	File Size
dog.jpg	2025-10-28	86.3 KB
.DS_Store	2026-03-31	6.00 KB
solitaire.exe	2025-10-28	116 KB
rubiks.jpg	2025-10-28	143 KB
computer.jpg	2025-10-28	575 KB
ocean.jpg	2025-10-28	41.9 KB

Figure 6 image.zip files

- Objective : Investigate a suspicious executable file.
- Tool Used : file
- Command : **file solitaire.exe**

≥ POC :

```
┌──(kali㉿kali)-[~/Desktop/image]
└─$ file solitaire.exe
solitaire.exe: PNG image data, 640 x 449, 8-bit/color RGBA, non-interlaced
```

Figure 7 File Signature Verification Result

≥ Analysis :

This file represents a normal fraudulent technique which is using a highly suspicious **.exe** executable extension to fraud the user or the systems. However, after deploying a **file** command, we can ignored the misleading filename and direct query the internal file signature. The output successfully exposed the deception, revealing that the structure is a standard PNG (Portable Network Graphics) image file. This finally put the solitaire.exe a harmless image files.

Target 4 : Rubiks.jpg

image.zip > **image**

Name	Date Modified	File Size
dog.jpg	2025-10-28	86.3 KB
.DS_Store	2026-03-31	6.00 KB
solitaire.exe	2025-10-28	116 KB
rubiks.jpg	2025-10-28	143 KB
computer.jpg	2025-10-28	575 KB
ocean.jpg	2025-10-28	41.9 KB

Figure 8 image.zip files

- Objective : Verify the true image format
- Tool Used : file
- Command : **file rubiks.jpg**

≥ POC :

```
┌──(kali㉿kali)-[~/Desktop/image]
└─$ file rubiks.jpg
rubiks.jpg: PNG image data, 609 x 640, 8-bit/color RGBA, non-interlaced
```

Figure 9 File Scans Result

≥ Analysis :

The file is labelled with a .jpg extension, which highlights a lossy compressed file. However, after executing the file command to interrogate its magic bytes, the system automatically identified it as PNG image data. This is a common method to obscure a true file format. Although it is different files extensions, doesn't mean it is a true files as trusted.

Target 5 : Dog.jpg

image.zip > **image**

Name	Date Modified	File Size
dog.jpg	2025-10-28	86.3 KB
.DS_Store	2026-03-31	6.00 KB
solitaire.exe	2025-10-28	116 KB
rubiks.jpg	2025-10-28	143 KB
computer.jpg	2025-10-28	575 KB
ocean.jpg	2025-10-28	41.9 KB

Figure 10 image.zip files

- Objective : Scan for embedded payloads within the image structure
- Tool Used : binwalk
- Command : **binwalk dog.jpg**

≥ POC :

```
┌──(kali㉿kali)-[~/Desktop/image]
└─$ binwalk dog.jpg

DECIMAL       HEXADECIMAL     DESCRIPTION
--------------------------------------------------------------------------------
0             0x0             JPEG image data, JFIF standard 1.01
88221         0x1589D         Zip archive data, at least v1.0 to extract, compressed size: 21, uncompressed size: 21, name: hidden_text.txt
88384         0x15940         End of Zip archive, footer length: 22
```

Figure 11 Binwalk dog.jpg

≥ Analysis :

This target required an advanced extraction technique since we need to identify a file which is hidden entirely by another file within the structure. By using a **binwalk** scan, we detected a significant abnormal of secondary data such as Zip archive. Plus, the tool also automatically carved out the **hidden flags**.

Summary of Findings

Table 1 Labs Summary

File	Tools	Command	POC	Analysis
ocean.jpg	exitfool	exiftool ocean.jpg	**Figure 2**	The hidden flag finally discovered in the metadata comment section
computer.jpg	hexeditor/strings	hexeditor computer.jpg strings computer.jpg	**Figure 4** **Figure 5**	Document header analysis confirmed the file is legitimate JPEG bytes
solitaire.exe	file	file solitaire.exe	**Figure 7**	file commands exposed the executable disguise PNG image
rubiks.jpg	file	file rubiks.jpg	**Figure 9**	file commands exposed the executable disguise PNG image
dog.jpg	binwalk	binwalk dog.jpg	**Figure 11**	Identified successfully the hidden file embedded within the image

Conclusion

This labs exercise has successfully demonstrated the untrusted default extensions within an operating systems. By applying a standard vulnerability analysis tools in a Linux in Virtual Box, various deception techniques can be prevented.

Whether it is uncovering the data in the metadata, proving the true identity of a file is must via magic bytes, or extracting nested files with Binwalk, these techniques are important to identify hidden information and mitigating the risk of unknown files.